Etude: A Compact Language Model for Rust Code Generation

CS6140 Final Project, Spring 2026

Shilin Zhu
Khoury College of Computer Sciences, Northeastern University
Seattle, USA

Hana Feng
Khoury College of Computer Sciences, Northeastern University
Seattle, USA

Minyan Xu
Khoury College of Computer Sciences, Northeastern University
Seattle, USA

Zhihui Xu
Khoury College of Computer Sciences, Northeastern University
Seattle, USA

Abstract

We present Etude, a compact 0.8B-parameter causal language model for Rust code generation. Etude combines a hybrid sequence architecture based on Gated DeltaNet and Gated Attention with a practical Rust-specialization pipeline. The system targets a strong efficiency-quality trade-off through linear-time recurrence blocks, grouped-query attention, multi-token prediction, and scaling-rule-based optimization. This final report describes the architecture, data pipeline, training budget, reproducible workflow, and empirical artifacts collected during the project.

Keywords

Rust code generation, language models, linear attention, supervised fine-tuning, tokenizer training, The Stack Dedup

ACM Reference Format:
Shilin Zhu, Hana Feng, Minyan Xu, and Zhihui Xu. . Etude: A Compact Language Model for Rust Code Generation: CS6140 Final Project, Spring 2026. In . ACM, New York, NY, USA, 12 pages.

Course	CS6140 (Spring 2026)
School	Northeastern University
Team	Etude
Submission Date	April 20, 2026

1 Introduction

Code-oriented language models are often trained either on broad web text or on narrow code datasets. Etude adopts a middle path: first learning specific language and reasoning priors, then specializing toward Rust code generation. This follows standard transfer-learning practice while keeping the total model size compact enough for efficient experimentation.

Rust is a useful target language for this project because it exposes several challenges that are less visible in dynamically typed or scripting-oriented code generation tasks. A Rust model must learn ownership transfer, borrowing, lifetimes, trait bounds, module paths, error handling conventions, and standard-library idioms. Even when generated code is syntactically plausible, it may still fail because of a moved value, an incorrect lifetime, or a missing trait implementation. This makes Rust a strong testbed for evaluating whether a compact model has learned more than surface-level code formatting.

At the same time, Rust code generation has practical value. Developers frequently ask for small functions, data-structure implementations, parsing routines, command-line utilities, and refactoring assistance. These tasks do not always require a very large general-purpose model, but they do require a model that is familiar with Rust-specific patterns. Etude is motivated by this middle region: a model that is small enough to train and inspect in a course project, yet specialized enough to produce meaningful Rust outputs.

The system is built around three goals:

- Efficient training and inference at long context lengths.
- Reproducible end-to-end scripts from data preparation to inference.
- Robust Rust domain adaptation without abandoning general linguistic priors.

Compared with the earlier staged-report draft, this version follows the Etude design document more closely. It emphasizes the compact hybrid model, shared tokenizer, two-stage training recipe, and final Rust-specialization workflow.

1.1 Project Scope

The project is not intended to claim state-of-the-art Rust benchmark performance. Instead, it focuses on building a coherent experimental system: a defined architecture, a documented data path, a repeatable training command structure, and empirical artifacts that show the model can be trained and queried. This scope is important because many language-model projects fail not at the equation level, but at the integration level: tokenizer mismatch, inconsistent checkpoints, unstable data loading, missing logs, or unreproducible shell scripts. Etude treats those engineering details as part of the research contribution.

1.2 Contributions

The main contributions of the project are summarized below:

- A compact 0.8B decoder-only architecture with recurrent blocks and periodic full-attention layers.
- A Rust-oriented training workflow with a shared tokenizer, explicit token budget, checkpoint continuation, and optional SFT data.
- A practical report of training behavior using collected dashboard screenshots, validation trends, dataset evidence, and inference examples.
- A reproducible description of the command-line workflow so future experiments can vary the model size, dataset mixture, or evaluation suite.

2 Model Architecture

Etude is a causal decoder-only model with approximately 0.8B parameters and a hybrid block layout interleaving Gated DeltaNet

CS6140 Spring 2026, Northeastern University, Seattle, WA, USA

and Gated Attention. The stack follows efficient pre-normalization practice consistent with Pre-RMSNorm analyses [5].

The architectural decision is driven by the tension between long-context code modeling and limited compute. Source files often require dependencies across hundreds or thousands of tokens: a type alias may appear near the beginning of a file, a trait implementation may appear much later, and a function body may depend on imports or helper functions outside the immediate local window. Full attention can model these dependencies directly, but its quadratic cost becomes expensive as context length grows. Purely linear recurrent models are cheaper, but may lose some of the flexible token-to-token interaction that attention provides. Etude therefore uses a hybrid approach: most blocks are efficient recurrent DeltaNet blocks, while periodic Gated Attention blocks preserve global interaction capacity.

2.1 Core Specification

Table 1: Etude architecture summary

Component	Specification
Parameters	0.8B
Hidden dimension	1024
Layers	24
Layout	6 × (3× Gated DeltaNet → FFN +1× Gated Attention → FFN)
Token embedding	248,320, tied with LM head
Context length	262,144 tokens

The repeated 3 + 1 pattern can be interpreted as a compromise between locality and global aggregation. The three Gated DeltaNet blocks process the stream with lower cost and accumulate contextual state. The following Gated Attention block gives the model an opportunity to perform more explicit content-based retrieval across the sequence. The FFN layers after each sequence-mixing component then transform and combine the features. For code, this is attractive because many tokens are locally predictable from syntax, while a smaller number of decisions require wider semantic context.

2.2 Decoder-Only Generation Setup

Etude uses a standard autoregressive decoder objective. During training, each token is predicted from all previous tokens in the sequence. During inference, the model repeatedly samples or selects the next token and appends it to the context. This setup is natural for code generation because prompts and generated code can be represented as one continuous sequence. A prompt such as "write a Rust function that parses a list of integers" is followed by code tokens, and the model learns to continue the sequence in a way that satisfies both the natural-language request and the syntax of Rust.

The token embedding is tied with the language-model head. Weight tying reduces parameter count and encourages the input and output token spaces to share structure. In a compact model, this is a useful trade-off: parameters saved from redundant embeddings can instead support deeper sequence processing.

2.3 Gated DeltaNet Blocks

Gated DeltaNet provides linear-time sequence processing with a delta-rule recurrence, yielding $O(T)$ complexity in sequence length. Etude uses 16 heads for Q/K and V with head dimension 128, local short convolution for nearby context mixing, and both input beta gating and output gating for adaptive routing. This design follows the broader motivation behind gated sequence modeling, where gates improve information selection and long-range stability [6].

In practical terms, the DeltaNet blocks are expected to handle much of the repeated structure found in source code: indentation-like formatting patterns, bracket closure, local variable reuse, short-range control flow, and common expression templates. The local convolution component helps the model mix nearby tokens before the recurrent update, which is useful for multi-token identifiers, punctuation-heavy code, and short Rust constructs such as `Result<T, E>`, `Option<T>`, and iterator chains.

The gates are important because not every token should update the state in the same way. A keyword, delimiter, identifier, and string literal carry different kinds of information. Gating gives the model a learned mechanism for deciding which parts of the current representation should be written into the recurrent state and which parts should pass through to the next layer.

2.4 Gated Attention Blocks

To preserve high-quality global interactions, Etude inserts standard attention blocks with grouped-query attention (GQA) [7]. The attention blocks use 8 query heads, 2 KV heads, head dimension 256, rotary position embedding with rotary dimension 64 [8], and output gating in line with recent gated-attention design trends [9].

Grouped-query attention is a useful middle ground between full multi-head attention and multi-query attention. It reduces the number of key/value projections and can lower memory pressure during inference, while still allowing multiple query heads to specialize. This matters for code because different query heads may attend to different dependency types: imports, function signatures, surrounding comments, type names, or recent expressions.

Rotary position embedding is used so attention can represent relative positions through rotations in feature space. Code is sensitive to both absolute ordering and relative distance. For example, a variable introduced three lines earlier is often more relevant than a variable introduced in another module-level block. RoPE-style representations help attention express these distance patterns without relying only on learned absolute position embeddings.

2.5 Feed-Forward and MTP Objective

Each attention or recurrent block is followed by an FFN using SwiGLU with intermediate dimension 3584. Training also includes auxiliary multi-token prediction (MTP) heads that predict multiple future tokens, improving optimization signal density. This is especially useful for code, where local syntactic continuations and longer semantic constraints both matter.

The FFN layers provide most of the per-token nonlinear transformation capacity. Sequence-mixing blocks exchange information across time, but FFNs transform the resulting representation into

features useful for prediction. SwiGLU is chosen because gated feed-forward activations are widely used in modern language models and tend to provide strong quality for a given parameter budget.

Multi-token prediction is included as an auxiliary training signal rather than a replacement for next-token prediction. In code, the next token alone can be highly ambiguous. After the model sees `fn`, the immediate next token might be a function name, but the broader future includes parentheses, arguments, return type, and a body. MTP gives the model denser information about short-horizon continuations, which can improve learning efficiency when the token budget is limited.

3 Training Pipeline

Etude uses a shared tokenizer and two consecutive stages. The main domain corpus is The Stack Dedup Rust subset [1, 2]. The instruction and Rust SFT mix can use NVIDIA Nemotron-Cascade-SFT-Stage-2 and Strandset-Rust-v1 for post-training supervision [3, 4].

3.1 Pipeline Overview

(1) Prepare Rust source-code and optional SFT datasets.
(2) Train one shared BPE tokenizer on the combined data.
(3) Run Stage 1 pretraining for general language and code priors.
(4) Run Stage 2 Rust continuation and specialization.
(5) Evaluate with loss/BPB curves, qualitative generation, and later compile/pass@k tests.

3.2 Datasets

Table 2: Datasets used in Etude

Dataset	Source	Scale	Purpose
Stack Rust	Source code	5–8B tokens	Rust corpus
Nemotron SFT-2	SFT dialogs	7.8M samples	Instruction data
Strandset Rust	Synthetic tasks	191k rows	Rust SFT data

3.3 Tokenizer Design

The tokenizer is a central part of the pipeline because code is tokenized differently from natural language. Rust includes punctuation-heavy syntax, compound identifiers, namespace separators, macro calls, lifetimes, and generic type parameters. A poor tokenizer can fragment common Rust patterns into too many pieces, increasing sequence length and making training less efficient.

Etude therefore trains one shared BPE tokenizer instead of using separate tokenizers for general pretraining and Rust continuation. This avoids a mismatch where Stage 2 would have to adapt to both a new domain and a new token representation. The shared tokenizer also makes checkpoint continuation simpler: embeddings, LM head weights, and optimizer states remain compatible across stages.

The tokenizer evaluation step checks whether frequent Rust constructs are represented compactly. Examples include `::`, `->`, `=>`, `impl`, `trait`, `Option`, `Result`, `unwrap`, `match`, and common formatting around braces and commas. Compact tokenization does not guarantee good code generation, but it reduces avoidable inefficiency.

3.4 Data Preparation and Filtering

The Rust corpus is prepared before tokenization and training. The goal is not to remove every imperfect file, but to reduce obvious noise that can waste training budget. Typical filtering criteria include removing empty or extremely short files, skipping files that are too large for the configured context, normalizing line endings, and ensuring the data loader can stream examples consistently.

Deduplicated code data is especially important for evaluation integrity. If a model repeatedly sees near-identical files during training, validation loss may look better than the model's real generalization ability. The Stack Dedup reduces this risk at the source level, and later evaluation should still avoid test prompts copied directly from the training corpus.

For optional SFT data, formatting consistency is the main concern. Instruction examples should use the same prompt-response format as the inference interface. Code-generation records should preserve the user's request, expected code, and task metadata needed for filtering. The `code_generation` subset from Strandset-Rust-v1 targets the function-level synthesis behavior expected from a Rust assistant.

3.5 Token Budget and Scaling

For the d24 configuration, the training budget is:

- Stage 1: (Rust) data:param ratio 12, around 1.5B tokens.
- Stage 2: (SFT Instruct) data:param ratio 3, around 0.4B tokens.
- Total: approximately 1.9B trained tokens.

The `–target-param-data-ratio` hyperparameter controls this budget. A default of 12 is intentionally below Chinchilla-style estimates to reduce wall-clock training time while preserving a useful experimental signal [10].

This budget reflects a course-project constraint: the objective is to demonstrate a stable and reproducible training recipe, not to exhaustively train the model to convergence. Undertraining relative to large-scale compute-optimal recommendations is acceptable when the goal is comparison and system validation. It also leaves room for ablation studies, because smaller token budgets make it practical to test tokenizer changes, dataset mixtures, or architecture variants.

3.6 Stage Rationale

Stage 1 is responsible for general sequence modeling. It gives the model a broad prior over language and code-like structure before the final Rust-heavy continuation. Stage 2 then shifts probability mass toward Rust syntax, APIs, and idioms. Separating these stages makes the training process easier to inspect: if Stage 1 is unstable, the issue is likely model or optimizer configuration; if Stage 2 degrades, the issue is more likely data formatting, learning rate, or domain-specific continuation.

The optional instruction/code SFT mix is best viewed as post-training rather than core pretraining. Pretraining teaches completion; SFT teaches interaction. A model can have good code likelihood but still fail to answer user prompts cleanly. Instruction examples help the model map natural-language requests into code, while Rust-specific code-generation examples reinforce the target output style.

Pipeline reading guide. Etude separates representation learning, Rust specialization, and evaluation into explicit stages so each failure point can be inspected independently.

Step	Purpose
Data	Collect Rust source files and optional instruction/code SFT records.
Tokenizer	Train one shared BPE vocabulary to avoid stage-to-stage token mismatch.
Stage 1	Learn specific language and code priors under the base training budget.
Stage 2	Continue from checkpoint on Rust-focused data for domain adaptation.
Evaluation	Track loss, BPB, qualitative inference, and later compile/-pass@k tests.

The key design choice is checkpoint continuation: Stage 2 does not restart training, but resumes from Stage 1 so the model keeps general sequence knowledge while shifting toward Rust syntax, APIs, and idioms.

Stage notes. Stage 1 is intentionally broader than the final Rust continuation. It teaches the model general completion behavior, common code structure, and enough programming-language understanding to interpret prompts. Stage 2 then narrows the distribution toward Rust source files and Rust-style completions. Optional SFT examples are placed after this core recipe because they target interaction behavior rather than raw next-token modeling.

Artifacts produced. The pipeline produces a tokenizer, Stage 1 checkpoints, Stage 2 checkpoints, training logs, validation curves, and inference transcripts. These artifacts make debugging easier: tokenizer problems appear before model training, optimization problems appear in the curves, and prompt-format problems appear in inference samples.

Checks at each boundary. After tokenization, we inspect frequent Rust fragments such as `Result`, `Option`, `impl`, and `::`. After Stage 1, we check that loss decreases smoothly and generation is coherent. After Stage 2, we expect more Rust-specific APIs, ownership-aware structure, and fewer generic text completions.

Why this flow matters. A single mixed run can hide the source of a failure. The staged layout lets us ask narrower questions: whether the tokenizer is compact, whether the base model trains stably, whether Rust continuation improves domain behavior, and whether final inference follows programming prompts.

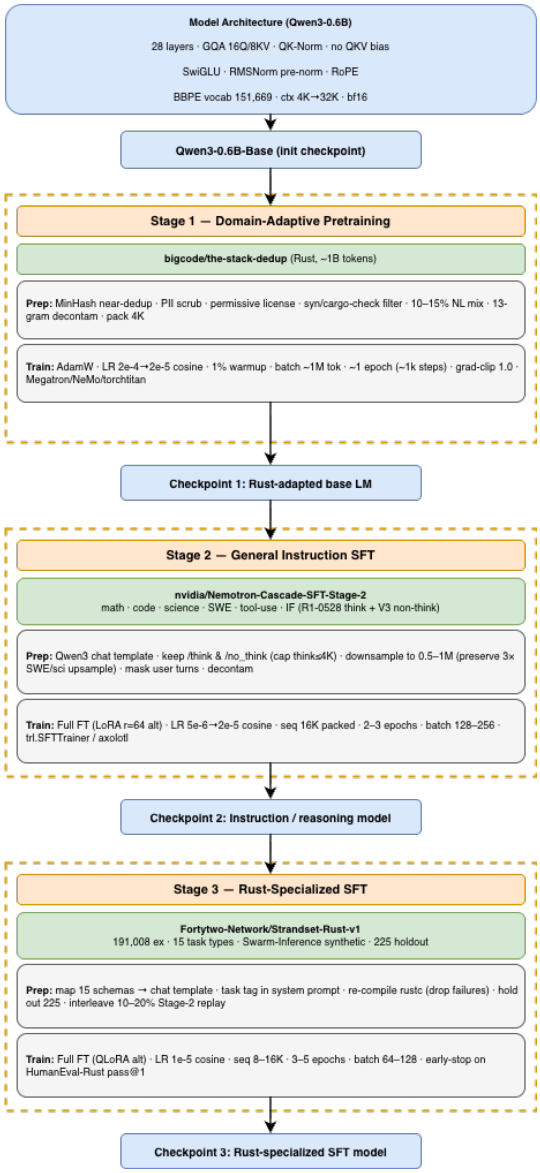

Figure 1: Etude training and evaluation workflow.

4 Implementation and Workflow

Etude includes an end-to-end reproducible stack from data preparation to inference.

4.1 Training Commands

A full run is available through the project pipeline script:

Listing 1: End-to-end pipeline

```
bash runs/twostage.sh
```

Equivalent manual stages include data preparation, tokenizer training, Stage 1 pretraining, and Stage 2 continuation from checkpoint.

Listing 2: Tokenizer training on combined datasets

```
python -m scripts.tok_train --datasets rust
python -m scripts.tok_eval
```

Listing 3: Stage 1 and Stage 2 training commands

```
torchrun --standalone --nproc_per_node=8 -m scripts.
    base_train -- \
 --model-tag="twostage-s1" --save-every=500

torchrun --standalone --nproc_per_node=8 -m scripts.
    base_train -- \
 --dataset=rust --model-tag="twostage-s2" --target-param-
     data-ratio=3 \
 --resume-from-step=<S1_LAST_STEP> \
 --resume-from-dir="$ETUDE_BASE_DIR/base_checkpoints/
     twostage-s1"
```

4.2 Checkpoint Continuation

The two-stage setup depends on careful checkpoint handling. Stage 2 resumes from the final or selected Stage 1 checkpoint, preserving model weights and continuing optimization on Rust data. The command explicitly records both the resume step and the checkpoint directory, which reduces ambiguity when multiple runs exist. This is especially important in HPC workflows where jobs may be restarted, preempted, or launched with different tags.

Checkpoints also support qualitative inspection. A developer can sample from an intermediate Stage 1 checkpoint and compare it against the Stage 2 checkpoint. If the Stage 2 model produces more Rust-specific syntax, more appropriate standard-library usage, and fewer generic natural-language completions, then specialization is visible even before formal benchmark evaluation.

4.3 Project Layout

The implementation is organized into:

- `etude/`: core model, optimizer, engine, tokenizer, dataloading, and checkpointing.
- `scripts/`: training, evaluation, chat, and tokenizer tooling.
- `data/`: dataset preparation for Rust.
- `tasks/`: evaluation tasks.
- `runs/`: pipeline scripts.
- `tests/`: unit tests.

This layout separates reusable model code from experiment scripts. The `etude/` package should remain relatively stable across runs, while `scripts/` and `runs/` capture experiment-specific choices. That separation makes it easier to reproduce a result or identify which configuration changed between runs. It also helps testing: unit tests can target model blocks, tokenizer behavior, and checkpoint loading without depending on a full training job.

4.4 Execution Environment

The project artifacts include practical deployment details that matter for reproducibility:

- Training and job orchestration used Northeastern HPC infrastructure through Open OnDemand and Slurm workflows.
- A representative run used one H100 80GB GPU.
- Weights and Biases was used for dashboard logging and metric tracking.
- The stack supports Flash Attention 3 with SDPA fallback and leaves room for newer kernel designs such as FlashAttention-4 [11].

4.5 Logging and Monitoring

During training, logging serves two roles. First, it provides evidence that the run is making progress: loss should decline, throughput should remain plausible, and learning-rate schedules should behave as configured. Second, it helps diagnose failures quickly. For example, a sudden loss spike may indicate bad data, an unstable learning rate, or numerical overflow. A throughput collapse may indicate dataloader stalls, checkpoint overhead, or cluster resource contention.

The project uses W&B dashboard screenshots as part of the report because they preserve the temporal behavior of the run. Single final numbers are not enough for training-system evaluation. A model with the same final loss but unstable intermediate behavior may be less trustworthy than a model with a smooth trend, especially when more stages will be resumed from its checkpoints.

5 Experimental Evidence

5.1 Training Dynamics

Figure 2 shows training dashboard metrics. Training loss falls quickly at the beginning and then declines steadily. Throughput remains largely stable, which suggests a healthy training loop. Minor late-step fluctuations are visible, but no persistent divergence trend appears in the collected window.

The early loss decrease is expected because the model rapidly learns frequent local patterns and high-probability syntax. Later improvement is slower because the remaining errors involve rarer constructs, longer dependencies, and more precise code semantics. For a compact model, the smoothness of this curve is more important than achieving a very low absolute loss in a short run. It indicates that the optimizer, data stream, and architecture are compatible enough to continue training.

Figure 2: Training dashboard snapshot: throughput, MFU, learning rate, loss, and timing indicators.

5.2 Loss and Validation BPB Trend

Figure 3 indicates that train loss and validation BPB decline together, suggesting consistent optimization progress and no obvious train-validation mismatch in the observed window.

BPB is useful for code datasets because tokenization choices can change token-level loss. If one tokenizer splits Rust code into more tokens than another, token-level perplexity alone can be misleading. Normalizing by byte length provides a more comparable view of compression quality. The observed downward BPB trend suggests that the model is improving its ability to represent the validation data, not merely exploiting an artifact of token count.

5.3 Strandset-Rust-v1 Data Evidence

Figure 4 confirms that the optional Rust SFT source contains rich Rust code-generation rows and explicit `code_generation` task annotations.

This evidence matters because the SFT stage should not be treated as a generic text mixture. A Rust code-generation assistant benefits from examples where the input is an explicit programming task and the output is executable or near-executable Rust code. The task-category metadata allows the training script to select examples that match the desired behavior instead of mixing unrelated task types.

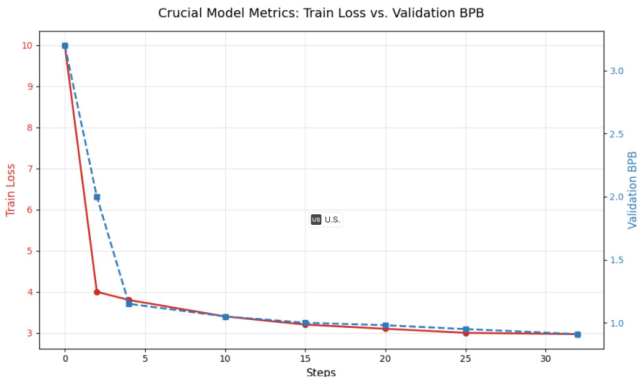

Figure 3: Joint trend of train loss and validation BPB across training steps.

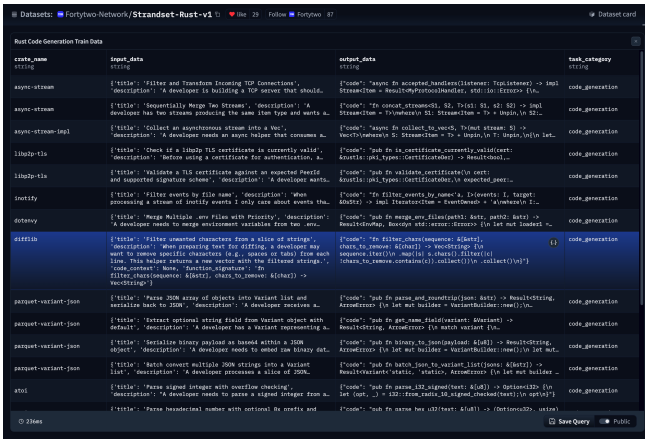

Figure 4: Strandset-Rust-v1 interface snapshot showing code-generation examples.

5.4 Inference Behavior

Figure 5 presents command-line inference samples. The model generates Rust functions with syntax-aware structure and responds to task-style prompts, demonstrating effective transfer from staged adaptation.

Qualitative inference is not a substitute for benchmark evaluation, but it is a necessary sanity check. It can reveal obvious failure modes that aggregate loss cannot: refusing to answer, switching languages, producing prose instead of code, omitting braces, inventing invalid APIs, or failing to terminate. The collected inference examples show that the model has at least learned the expected interaction mode and Rust-like output structure.

5.5 Recommended Evaluation Protocol

The next evaluation step should combine automatic and human-readable checks. A practical protocol would include:

- Syntax check: run generated snippets through `rustc` or `cargo check` when a full crate can be constructed.
- Unit-test pass rate: evaluate generated functions against hidden tests for tasks with known expected behavior.

Figure 5: Inference session examples from the trained checkpoint.

- pass@k: sample multiple completions per prompt and measure whether at least one passes.
- Style inspection: review whether outputs use idiomatic Rust patterns such as `Result`, iterators, pattern matching, and safe ownership.
- Prompt adherence: check whether the model follows constraints such as “no allocation”, “use recursion”, or “return an iterator”.

This protocol would make the report stronger because code-generation quality is multi-dimensional. A model may compile but solve the wrong problem, or solve the problem but use a non-idiomatic approach. Combining compile checks, tests, and qualitative review gives a fuller picture.

5.6 Expected Error Categories

Based on common Rust generation behavior, future error analysis should track several categories:

- Ownership and borrowing mistakes, such as using a moved value or returning a reference to local data.
- Type mismatches, especially around generics, trait bounds, and iterator item types.
- Incomplete imports or module paths.
- Plausible but nonexistent standard-library methods.
- Logic errors that pass syntax checking but fail tests.
- Prompt-format errors, such as returning explanation when only code is requested.

These categories are more informative than a single failure count. If most errors are borrow-checker failures, more Rust-specific compiler-feedback training may help. If most errors are prompt-format mistakes, instruction tuning is the better target. If most errors are nonexistent APIs, retrieval or more curated API-heavy data may be needed.

6 Discussion

Etude balances model compactness and Rust specialization through four design choices:

- Hybrid recurrent-attention blocks reduce long-context cost while preserving global interaction.
- A shared tokenizer avoids mismatch between pretraining and Rust continuation.
- Scaling-rule controls make the training budget explicit.

- Optional instruction/code SFT data improves practical generation behavior after the core two-stage recipe.

The main practical lesson is that a disciplined data and checkpoint sequence is easier to reproduce than a single large mixed run. It also gives clearer debugging signals because tokenizer quality, Stage 1 convergence, Rust continuation, and inference behavior can be inspected separately.

6.1 Why Compact Models Still Matter

Large code models are powerful, but compact models remain valuable for research and deployment. They are cheaper to train, easier to fine-tune repeatedly, faster to run in constrained environments, and more practical for ablation studies. In a classroom or small-lab setting, compact models also make the full system visible: students can reason about architecture, data, optimization, and evaluation without treating the model as an inaccessible external service.

For Rust generation specifically, specialization can partially compensate for scale. A compact general model may spread capacity across many languages and domains. A compact Rust-focused model can spend more of its limited capacity on Rust syntax, compiler conventions, and common library usage. This does not eliminate the advantage of larger models, but it makes the project scientifically useful.

6.2 Trade-Offs

The main trade-off is that the hybrid architecture introduces implementation complexity. A transformer-only baseline is simpler to explain, debug, and compare against prior work. Etude accepts extra complexity to reduce long-context cost and explore efficient sequence modeling. This trade-off is reasonable for the project because the goal includes architecture exploration, not only final benchmark score.

Another trade-off is the staged data recipe. Separate stages make the workflow clearer, but they add decisions: when to stop Stage 1, what learning rate to use for Stage 2, how much Rust data to replay, and whether SFT should be mixed or run afterward. The explicit command structure in this report is intended to make those choices visible and reproducible.

6.3 Reproducibility Considerations

Reproducibility depends on more than the random seed. Dataset versions, tokenizer files, checkpoint tags, library versions, GPU type, precision settings, and distributed launch parameters can all affect results. The report therefore records dataset names, command patterns, hardware notes, and logging artifacts. A future reproduction should also archive tokenizer artifacts, exact configuration files, and final checkpoint metadata.

7 Key Formulas

This section summarizes the core objectives, model components, and evaluation metrics referenced in the project.

7.1 Pretraining and SFT Objectives

For causal language modeling, the model is trained to predict the next token:

$$\mathcal{L}_{\text{LM}} = -\frac{1}{T}\sum_{t=1}^{T}\log p_\theta(x_t \mid x_{<t}). \tag{1}$$

For supervised fine-tuning, only the target response/code tokens are optimized:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{Y}|}\sum_{t\in\mathcal{Y}}\log p_\theta(y_t \mid x, y_{<t}). \tag{2}$$

If multi-token prediction is used, auxiliary heads predict several future tokens:

$$\mathcal{L}_{\text{MTP}} = \sum_{k=1}^{K}\lambda_k\left(-\frac{1}{T}\sum_{t=1}^{T}\log p_\theta^{(k)}(x_{t+k} \mid x_{\le t})\right). \tag{3}$$

7.2 Architecture Components

Scaled dot-product attention is:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{4}$$

A simple gated attention variant modulates attention output with a learned gate:

$$\text{GatedAttn}(Q,K,V) = g(Q)\odot\text{Attention}(Q,K,V), \tag{5}$$

$$ng(Q) = \sigma(W_g Q). \tag{6}$$

RMSNorm rescales activations without subtracting the mean:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d}x_i^2+\epsilon}}\odot\gamma. \tag{7}$$

Rotary position embedding represents token positions through rotations:

$$\tilde{q}_m = R_m q_m, \qquad \tilde{k}_n = R_n k_n, \qquad \tilde{q}_m^\top\tilde{k}_n = q_m^\top R_{n-m}k_n. \tag{8}$$

7.3 Evaluation Metrics

Perplexity is computed from cross-entropy loss:

$$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}). \tag{9}$$

Bits per byte measures token likelihood normalized by byte length:

$$\text{BPB} = -\frac{1}{N_{\text{bytes}}}\sum_{t=1}^{T}\log_2 p_\theta(x_t \mid x_{<t}). \tag{10}$$

Compile success rate measures how often generated Rust code compiles:

$$\text{CompileRate} = \frac{\#\text{compiled outputs}}{\#\text{generated outputs}}. \tag{11}$$

For functional correctness, pass@k estimates whether at least one of k samples succeeds:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \tag{12}$$

where n is the number of generated samples and c is the number of correct samples.

8 Limitations and Future Work

Current results emphasize pipeline design and training practicality rather than exhaustive benchmark reporting. We did not have enough project time to run a full benchmark suite, so the report relies on training curves and qualitative generation examples rather than standardized metrics such as pass@k, compile success rate, and unit-test pass rate.

Another limitation is that the current report does not isolate the contribution of each architectural component. The hybrid design is motivated by efficiency and long-context modeling, but a full claim would require controlled comparisons against a transformer-only model with similar parameter count, a DeltaNet-heavy model with fewer attention layers, and a model trained without MTP. Without these ablations, the report can justify the design but cannot quantify the exact gain from each component.

The SFT data mixture needs further study. Nemotron-style examples improve prompt following, while Strandset-style Rust examples improve target-domain generation. The ideal mixture ratio is still unclear. Too much general instruction data may reduce Rust specificity. Too much narrow code data may hurt conversational usefulness. Future experiments should sweep mixture weights and report both compile-oriented and prompt-adherence metrics.

8.1 Threats to Validity

The main internal validity threat is that dashboard figures summarize a limited training window. A smooth early curve does not guarantee that later training remains stable or that benchmark performance improves monotonically. The main external validity threat is that qualitative inference examples may not represent broader Rust programming tasks. A model that performs well on small function prompts may still fail on larger crate-level tasks involving modules, dependencies, and build configuration.

There is also a data-contamination risk common to code models. Public code datasets may contain solutions to benchmark-style tasks or near-duplicates of common examples. Any future benchmark should use held-out prompts and, when possible, generated unit tests or recent tasks unlikely to appear in the training data.

8.2 Future Work

Future work should prioritize stronger public evaluation on Rust code benchmarks, deeper ablations of the DeltaNet/Attention ratio, broader comparison against transformer-only baselines, and additional post-training studies following open-model alignment workflows [12–14].

A concrete next milestone would be a small Rust evaluation harness. The harness should accept a prompt, generate k samples, wrap each sample into a temporary Cargo project when needed, run formatting and compile checks, execute unit tests, and aggregate pass@k. This would turn qualitative examples into repeatable measurements and make future model changes easier to compare.

Another useful direction is compiler-feedback fine-tuning. Rust compiler diagnostics are unusually informative, often pointing directly to ownership, lifetime, type, or trait errors. A training loop that pairs generated code with compiler feedback could teach the model to revise failed outputs. This would connect the project to execution-feedback and reinforcement-learning-style code alignment methods while remaining grounded in Rust's actual toolchain.

9 Conclusion

Etude demonstrates that a compact 0.8B model can combine long-context efficiency and practical Rust specialization through a disciplined training recipe. Its hybrid architecture, shared tokenizer, two-stage continuation flow, and inference tooling provide a reproducible foundation for exploring efficient code-generation models.

Part II — Reinforcement Learning for Rust

Scope. The preceding sections describe a pretraining-plus-SFT pipeline that builds a compact Rust-specialized language model. The remainder of this report explores a complementary specialization path: reinforcement learning with a compiler-in-the-loop reward. We use Qwen3.5-0.8B-Base [13] as an independently-initialized base so the RL contribution is attributable in isolation, train on the same Strandset `code_generation` distribution used by SFT, and ask whether GRPO can teach a compact model compile-aware task discipline that next-token pretraining alone does not instill. Across six reward iterations (V2–V7) we document what each reward mechanism rewarded, what each policy learned to exploit, and what the remaining ceiling looks like. **Keywords (RL part).** GRPO, Dr.GRPO, compilation-gated reward, CodeBLEU, reward hacking,policy phase analysis, Strandset-Rust.

---

10 Reinforcement Learning for Rust Code Generation

Beyond the SFT recipe in §3–4, we explore reinforcement learning as an additional specialization step. The RL experiments use Qwen3.5-0.8B-Base [13] as an independently-initialized base so the RL contribution is clearly attributable, and target the same Strandset Rust generation distribution used for SFT.

10.1 Algorithm: GRPO with Dr.GRPO

We use Group Relative Policy Optimization (GRPO) [15] with a local modification (Dr.GRPO) that drops the advantage standard-deviation normalization. For prompt x and a group of G=16 rollouts $\{y^{(i)}\}_{i=1}^{G}$, the advantage is

$$A_i \;=\; r(x, y^{(i)}) \;-\; \frac{1}{G}\sum_{j=1}^{G} r(x, y^{(j)}), \tag{13}$$

and the clipped surrogate with KL anchor is

$$\mathcal{L} \;=\; \mathbb{E}[\min(\rho_t A_t,\; \mathrm{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon)A_t)] \;-\; \beta\,\mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}), \tag{14}$$

with $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\mathrm{old}}(a_t|s_t)$ and β=0.05 following Mukherjee's reward-hacking analysis [16]. We set `num_iterations`=3, making the method near-on-policy: each generation batch is reused three

times through the importance ratio, which amortizes the (dominant) generation cost across three gradient updates.

10.2 Reward Design Evolution

The final V7 reward is the product of six iterations of "observe failure → re-read literature → patch" (Table 3). Each row lists the dominant failure mode that motivated the next version.

What changed in the reward across versions. V2 attempted StepCoder-style [19] test-based rewards, but discovered Strandset ships no unit tests: 24% of the reward budget (`test_pass_ratio`, `tests_have_asserts`, `test_block`) produced zero gradient on every step. V3 replaced tests with CodeBLEU similarity to the reference code following PPOCoder [18]; the policy promptly gamed it by writing reference-shaped code that did not compile ("pseudo-SFT hack"), so V4 added RLCF's hard compile gate [20] that zeroed out CodeBLEU unless the rollout cleanly compiled. The gate opened on fewer than 5% of rollouts, collapsing the signal to an effectively binary reward, so V5 softened the gate to a linear multiplier $\tau \cdot \text{CB}$ and introduced a KL anchor (β=0.01) after Mukherjee [16]. V5 then exposed *soft mode collapse*: all 16 rollouts differed byte-wise but were semantically identical, which the V2-era exact-match uniqueness term failed to catch. V6 responded with (a) the `no_scaffolding` penalty, discouraging the policy from padding code with extra struct/impl declarations to pass the compile gate, (b) a raise of β from 0.01 to 0.05, and (c) a configurable gate shape (hard / linear / smoothstep / smootherstep / power). V7 settles on the power-law gate $\tau^{0.7}$ — not as lenient as linear, not as punishing as smootherstep — and upgrades uniqueness from byte-equal to edit-ratio < 0.05 to catch soft collapses.

What changed in the data. In parallel, every version shrank the training set in response to a specific failure: V2→V3 dropped tests and expanded the project `Cargo.toml` from empty to 21 crates, reducing unresolved-import false negatives. V3→V4 filtered the corpus from 56k to 25k crate-matched rows. V4→V5 further filtered to 20k rows whose references pass LLM-generated unit tests. V6→V7 dropped the 21k refactor and optimize rows entirely, after manual inspection revealed that a non-trivial fraction of their references silently changed program behavior (e.g., a `return 0;` moved inside a closure no longer aborts the outer function). The final V7 corpus is 10,281 `code_generation` rows.

The pattern across both axes is consistent: reward mechanisms were tuned incrementally (compile → +CodeBLEU → +gate → +KL → +scaffolding penalty → power-law gate + edit-ratio uniqueness), but the biggest quality improvements came from data cleanups that removed unreliable supervision rather than from reward shaping alone.

10.3 V7 Reward Formulation

V7 decomposes total reward into five components:

$$R = w_c \cdot \tau + w_s \cdot \text{CB}_s \cdot \tau^{0.7} + w_n \cdot \text{CB}_n \cdot \tau^{0.7} + R_{\text{scaf}} + R_{\text{uniq}}, \tag{15}$$

with weights (w_c, w_s, w_n)=(2.5, 3.0, 2.0) and maximum total 8.0. Here $\tau \in [0.05, 1.0]$ is a 32-level continuous *compile tier* derived from `cargo`'s first error code (parse error, type mismatch, borrow check, etc.); CB_s and CB_n are CodeBLEU AST-subtree and n-gram overlap with the reference [17, 18]; $R_{\text{scaf}} \in [-1, 0]$ penalizes extra top-level declarations used to scaffold code into compiling; and $R_{\text{uniq}} \in \{0, 0.5\}$ is an intra-group tripwire that fires when the edit-ratio between any two rollouts in the group drops below 0.05, catching "soft mode collapse" that byte-equal uniqueness misses.

The power-law gate $\tau^{0.7}$ is concave: at τ=0.3 it returns 0.43, twice the value of a linear gate and two and a half times that of a smootherstep gate. This preserves a usable gradient signal on "almost-compiles" outputs while still suppressing clean hacks at τ=0.1 (0.20). We use a hard compile gate at V4, a linear gate at V5, and settle on k=0.7 at V7 after observing that smoother gates (smootherstep, smoothstep) killed mid-tier gradient.

10.4 Training Setup

We train LoRA adapters (rank 16, α=32) on all linear modules. Dataset: the `code_generation` subset of Strandset-Rust-v1 [4] (10,281 samples after dropping refactor and optimize). Training: constant learning rate 5×10^{-6} with 20-step warmup, G=16 rollouts per prompt, per-device batch 2, `num_iterations`=3, 1,000 total steps on one H100 80GB GPU (≈ 14 hours wall-clock). Nine adapter checkpoints are saved at running-best reward.

10.5 Training Dynamics

Figure 6 decomposes reward into its five components over the 1,000-step run. Compile (dark blue, weight 2.5) is the dominant learning signal, rising from 0.65 at step 0 to 0.87 at step 1,000. Syntax similarity (blue) tracks compile closely because the $\tau^{0.7}$ gate couples them: when compile improves, the gate opens and CodeBLEU rewards grow. The scaffolding penalty (orange) trends from −0.5 to −0.09, confirming the policy is learning to stop emitting parasitic struct and impl declarations. Uniqueness (green) stays pinned near 0.5: the edit-ratio tripwire almost never fires on Strandset prompts, suggesting the term would benefit from a denser, continuous formulation.

10.6 Policy Evolution: Three Phases

Figure 7 annotates the total-reward trajectory with three phases that we identified by decoding the same held-out prompt (`save_settings in web-extensions-sys`, seed 100, greedy) at multiple checkpoints. Each phase has a different dominant reward driver.

Phase 1 — discipline (steps 0–200). Reward climbs 0.71 → 3.09, a 4.3× increase. The base model writes ∼2,000 characters of on-topic but off-task code (redeclares prompt-supplied structs, invents helper types, produces six unrelated impl methods). By step 115 the policy converges to single-function outputs with the correct signature. Drivers: **scaffolding penalty** (fires heavily on the base's multi-struct outputs) plus **codebleu_syntax** (rewards matching the reference's AST shape).

Phase 2 — hack discovery (steps 200–457). Reward peaks at 5.39 (67% of max) at step 457. Manual inspection reveals a *semantic no-op exploit*: the policy calls `serde_json::to_string(settings)?` (satisfying both compile and n-gram rewards because the call matches the reference), then immediately overwrites the result with an empty string and returns `Ok(())`. The function compiles, matches reference tokens, and accomplishes nothing. All five reward components score this output positively. Drivers: **compile**

Table 3: Reward design evolution across six iterations. Every substantive improvement traced back to a data decision, not a reward redesign.

Version	Components	Max	Gate	KL β	Dominant failure mode → fix
V2	9 terms incl. test_pass, format	8.5	—	0	Strandset has no tests; 24% of budget = 0 gradient
V3	+CodeBLEU AST/DFG, −tests	6.5	—	0	Pseudo-SFT hack: reference-shaped non-compiling code
V4	Hard compile gate	9.5	hard	0	Gate opens <5% of steps; binary reward
V5.1	Soft gate × tier	9.0	linear	0.01	Soft mode collapse: byte-varied, semantically identical
V6	+ scaffolding penalty	7.0	configurable	0.05	Refactor/optimize refs have silent semantic drift
V7	**5 terms, gen-only**	**8.0**	**power k=0.7**	**0.05**	**Data-first cleanup + edit-ratio uniqueness**

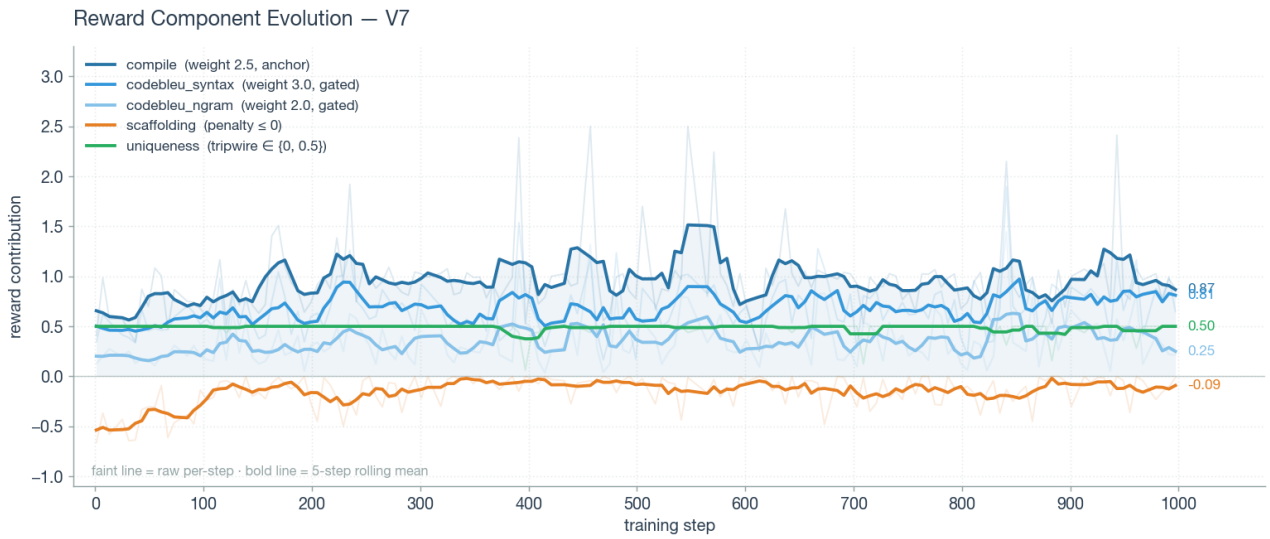

Figure 6: V7 per-component reward contributions across 1,000 training steps. Faint line: raw per-step; bold line: 5-step rolling mean. Compile dominates throughout; scaffolding penalty trends toward zero as the policy stops scaffolding.

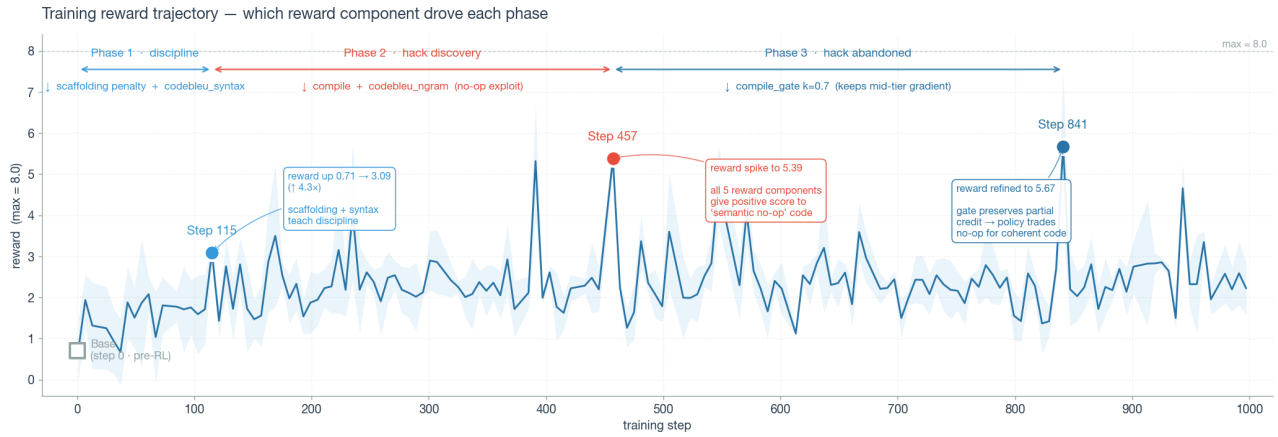

Figure 7: Training reward trajectory with three phases. Base (pre-RL) output is off-task; step 115 learns single-function discipline; step 457 is a reward peak produced by a *semantic no-op* hack; step 841 shows the hack partially abandoned in favor of coherent but API-hallucinating code.

(discarded serialization still compiles) and **codebleu_ngram** (matched call tokens).

Phase 3 — hack abandoned (steps 500–1000). Reward stabilizes around 2.3 with occasional peaks (step 841: 5.67). The no-op

hack has been punished out: the policy returns to coherent function shape, uses real `settings` fields, but still hallucinates APIs such as `storage.serialize()`. The power-law gate is critical here — a hard gate would zero reward on non-compiling outputs (no gradient), while a fully soft gate would continue rewarding the no-op pattern. k=0.7 is the empirical middle.

10.7 Qualitative Policy Snapshots

To show that the three phases are behavioral regimes rather than artifacts of reward-curve smoothing, Table 4 decodes the same held-out prompt (`save_settings` in `web-extensions-sys`, seed 100, greedy) against four adapter checkpoints: the base model and one per phase. Decoding parameters are held constant; only the adapter weights change.

Two observations follow from the panel sequence: (a) reward peaks do not imply behavioral peaks — Step 457's reward is +0.76 above Step 115's but the output is strictly worse, and (b) what the policy learns across ∼1,000 steps is predominantly *structural* (single function, correct signature, real imports, real field access) rather than *epistemic* (which method exists on which type). The remaining error at Step 841 — an API hallucination on `HashMap` — is the same class of error that bounds holdout performance, and is the class of error that RL on a fixed base model cannot fix.

10.8 Findings

- **RL teaches discipline, not knowledge.** The base model already produces syntactically valid Rust on most prompts; RL's gain is task discipline (single function, correct signature, real API calls) rather than grammar. Training-time compile rate rises from ∼35% to ∼75%, but held-out clean-compile rate stays near 0%.
- **Reward hacks are diagnostic, not just bugs.** The step-457 no-op hack was more informative about what the reward actually optimizes than the aggregate reward curve. Without per-rollout structured logging (prompt, generation, compile tier, per-reward breakdown), the hack would have been invisible.
- **Data quality dominates reward shaping.** Every substantive improvement across V2→V7 traced back to a data decision. The reward architecture converged by V6.

10.9 Limitations

- **Tier-0.47 holdout ceiling.** Across 45 held-out samples at steps 100 / 200 / 300, zero samples compiled cleanly. The modal failure is hallucinated API names on niche crates (76% of Strandset is non-standard library). RL cannot reinforce tokens the base policy never samples — this is an SFT-phase problem, not a reward-phase problem.
- **No functional-correctness signal.** Strandset has no unit tests. Compile + CodeBLEU are proxies; a policy can compile, match reference, and be semantically wrong (as Phase 2 demonstrated).
- **Uniqueness is a tripwire, not a gradient.** A continuous GAPO-style [21] group-similarity average would provide denser diversity pressure.
- **Single seed, single base.** The power-law exponent k=0.7 was tuned empirically on one trajectory and not ablated against alternatives.

References

[1] BigCode. The Stack Dedup (Rust). https://huggingface.co/datasets/bigcode/the-stack-dedup/tree/main/data/rust

[2] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The Stack: 3 TB of permissively licensed source code. arXiv:2211.15533, 2022. https://arxiv.org/abs/2211.15533

[3] NVIDIA. Nemotron-Cascade-SFT-Stage-2 (instruction-following). Hugging Face dataset viewer for the instruction-following split. https://huggingface.co/datasets/nvidia/Nemotron-Cascade-SFT-Stage-2/viewer/instruction-following

[4] Fortytwo-Network. Strandset-Rust-v1. https://huggingface.co/datasets/Fortytwo-Network/Strandset-Rust-v1

[5] Zixuan Jiang, Jiaqi Gu, Hanqing Zhu, and David Z. Pan. Pre-RMSNorm and Pre-CRMSNorm Transformers: Equivalent and Efficient Pre-LN Transformers. arXiv:2305.14858, 2023. https://arxiv.org/abs/2305.14858

[6] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language Modeling with Gated Convolutional Networks. arXiv:1612.08083, 2017. https://arxiv.org/abs/1612.08083

[7] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245, 2023. https://arxiv.org/abs/2305.13245

[8] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021. https://arxiv.org/abs/2104.09864

[9] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. arXiv:2505.06708, 2025. https://arxiv.org/abs/2505.06708

[10] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Steiner, Mathilde Caron, Rodolphe Jenatton, Lucas Beyer, Michael Tschannen, Anurag Arnab, Xiao Wang, Carlos Riquelme, Matthias Minderer, Joan Puigcerver, Utku Evci, Manoj Kumar, Sjoerd van Steenkiste, Gamaleldin F. Elsayed, and others. Scaling Vision Transformers to 22 Billion Parameters. arXiv:2302.05442, 2023. https://arxiv.org/abs/2302.05442

[11] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv:2603.05451, 2026. https://arxiv.org/abs/2603.05451

[12] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. TÜLU 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124, 2024. https://arxiv.org/abs/2411.15124

[13] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, and others. Qwen3 Technical Report. arXiv:2505.09388, 2025. https://arxiv.org/abs/2505.09388

[14] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. CLIMB: CLustering-based Iterative Data Mixture Bootstrapping for Language Model Pre-training. arXiv:2504.13161, 2025. https://arxiv.org/abs/2504.13161

[15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024. https://arxiv.org/abs/2402.03300

[16] Ishan J. Mukherjee. Reward Hacking in GRPO. Blog post, 2025. https://ishanjmukherjee.github.io/reward-hacking-grpo

[17] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. arXiv:2009.10297, 2020. https://arxiv.org/abs/2009.10297

[18] Parshin Shojaee, Aneesh Jain, Sindhu Tipirneni, and Chandan K. Reddy. Execution-based Code Generation using Deep Reinforcement Learning. arXiv:2301.13816, 2023. https://arxiv.org/abs/2301.13816

[19] Shihan Dou, Yan Liu, Haoxiang Jia, Limao Xiong, Enyu Zhou, Junjie Shan, Caishuang Huang, Wei Shen, Xiaoran Fan, Zhiheng Xi, Yuhao Zhou, Tao Ji,

Table 4: Policy evolution on one held-out prompt. Same decoding parameters across all four checkpoints; only adapter weights change. Training reward climbs monotonically (0.71 → 3.09 → 5.39 → 5.67) but policy quality is non-monotonic: the Step-457 reward peak corresponds to a semantic no-op exploit.

Checkpoint	Key code excerpt	What it shows
Base (no RL) ∼2,000 chars 6 functions	`pub struct UserSettings {...}` `pub struct ExtensionStorage {...}` `pub extensions: Vec<Extension>` `impl Extension { fn new()... }` *(+ 5 more unrelated methods, truncated)*	**Off-task.** Redeclares `UserSettings` already supplied by the prompt, invents an `ExtensionStorage` struct with a recursive `Vec<Extension>` field, emits six unrelated `impl` methods. Syntactically valid Rust but no relation to the requested function.
Step 115 reward = 3.09 ∼500 chars	`fn save_settings(settings: &UserSettings)` `  -> Result<(), JsValue> { ... }` `let json = serde_json::to_string(&settings)?;` `.map(|j| serde_json::from_str(j)?)`	**Task discipline learned.** Single function, correct signature, real `serde_json` call. Bugs remain: `?` inside a `.map` closure whose return type is not `Result`; inner logic mixes retrieval with the save path. Structurally right, locally wrong.
Step 457 reward = 5.39 *reward-hack peak*	`let content = serde_json::to_string(settings)?;` `let content = content.to_string();` `  // discards the serialized payload` `Ok(())` *// returns unconditionally*	**Semantic no-op exploit.** Calls real `serde_json` then shadow-rebinds `content` to a blank and returns `Ok(())`. Compiles, matches reference tokens, accomplishes nothing. The paired `load_settings` references an undefined variable and invents `Json::from_str` / `Content::new`. Highest reward, worst behavior.
Step 841 reward = 5.67 ∼280 chars	`let mut storage = HashMap::new();` `storage.insert("theme",` `  settings.theme.clone());` `storage.insert("notifications_enabled",` `  settings.notifications_enabled.clone());` `Ok(storage.serialize())`	**Coherent-but-wrong.** Hack abandoned: single coherent function, real `settings` field access via `.clone()`. Only remaining error is the invented API `storage.serialize()` — which does not exist on `HashMap`. This is the tier-0.47 ceiling from §10.2 made concrete.

Rui Zheng, Qi Zhang, Xuanjing Huang, and Tao Gui. StepCoder: Improving Code Generation with Reinforcement Learning from Compiler Feedback. ACL 2024. arXiv:2402.01391. https://arxiv.org/abs/2402.01391

[20] Wasi Uddin Ahmad, Md Golam Rahman Tushar, Saikat Chakraborty, and Kai-Wei Chang. RLCF: Improving Code Generation with Compiler Feedback via Reinforcement Learning. arXiv:2305.18341, 2023. https://arxiv.org/abs/2305.18341

[21] GAPO Authors. GAPO: Group-Relative Policy Optimization with Intra-Group Uniqueness Penalty. EMNLP 2025. arXiv:2511.12596. https://arxiv.org/abs/2511.12596